

NAME :Y.VARUN KUMAR REDDY
REG NO : 192424230

```
# Install Ultralytics
!pip install ultralytics -q

from ultralytics import YOLO
import cv2
import numpy as np
from sklearn.metrics import precision_score, recall_score

# YOLO model
model = YOLO("yolov8n.pt")

# Sample ground truth
y_true = [1, 1, 1, 0, 0]

# Example predictions
viola_pred = [1, 1, 0, 1, 0]
yolo_pred = [1, 1, 1, 0, 0]

print("Viola-Jones Precision:",
      precision_score(y_true, viola_pred))
print("Viola-Jones Recall:",
      recall_score(y_true, viola_pred))

print("YOLO Precision:",
      precision_score(y_true, yolo_pred))
print("YOLO Recall:",
      recall_score(y_true, yolo_pred))
```

```
===== 45.7/45.7 kB 786.6 kB/s eta 0:00:00
===== 1.4/1.4 MB 23.0 MB/s eta 0:00:00
===== 53.2/53.2 kB 1.5 MB/s eta 0:00:00
===== 64.2/64.2 kB 2.6 MB/s eta 0:00:00
```

Creating new Ultralytics Settings v0.0.7 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics>

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to 'yolov8n.pt': 100%  6.2MB 2

Viola-Jones Precision: 0.6666666666666666

Viola-Jones Recall: 0.6666666666666666

YOLO Precision: 1.0

YOLO Recall: 1.0

```
from sklearn.metrics import f1_score

y_true = [1, 1, 1, 0, 0, 1, 0, 1]

detector_A = [1, 1, 0, 0, 0, 1, 0, 1]
detector_B = [1, 0, 1, 1, 0, 1, 0, 1]

f1_A = f1_score(y_true, detector_A)
f1_B = f1_score(y_true, detector_B)

print("Detector A F1 Score:", f1_A)
print("Detector B F1 Score:", f1_B)

if f1_A > f1_B:
    print("Detector A performs better")
else:
    print("Detector B performs better")
```

Detector A F1 Score: 0.8888888888888888

Detector B F1 Score: 0.8

Detector A performs better

```
def calculate_iou(box1, box2):
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    intersection = max(0, x2-x1) * max(0, y2-y1)
```

```

area1 = (box1[2]-box1[0]) * (box1[3]-box1[1])
area2 = (box2[2]-box2[0]) * (box2[3]-box2[1])

union = area1 + area2 - intersection

return intersection / union

ground_truth = [50, 50, 200, 200]
prediction = [60, 60, 190, 190]

iou = calculate_iou(ground_truth, prediction)

print("IoU:", iou)

thresholds = [0.3, 0.5, 0.7, 0.9]

for t in thresholds:
    if iou >= t:
        print("Threshold", t, ": Correct Detection")
    else:
        print("Threshold", t, ": Incorrect Detection")

```

```

IoU: 0.7511111111111111
Threshold 0.3 : Correct Detection
Threshold 0.5 : Correct Detection
Threshold 0.7 : Correct Detection
Threshold 0.9 : Incorrect Detection

```

```

import time

# Example results
classical_accuracy = 0.75
deep_accuracy = 0.92

# Simulated processing time
start = time.time()
time.sleep(0.1)
classical_time = time.time() - start

start = time.time()
time.sleep(0.3)
deep_time = time.time() - start

print("Classical Detector")
print("Accuracy:", classical_accuracy)
print("Time:", classical_time)

print("\nDeep Learning Detector")
print("Accuracy:", deep_accuracy)
print("Time:", deep_time)

```

```

Classical Detector
Accuracy: 0.75
Time: 0.10054969787597656

Deep Learning Detector
Accuracy: 0.92
Time: 0.30039334297180176

```

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create first frame
frame1 = np.zeros((300, 300), dtype=np.uint8)
cv2.circle(frame1, (100, 150), 30, 255, -1)

# Create second frame with the object moved
frame2 = np.zeros((300, 300), dtype=np.uint8)
cv2.circle(frame2, (130, 150), 30, 255, -1)

# -----
# Sparse Optical Flow
# -----

points = cv2.goodFeaturesToTrack(
    frame1,
    maxCorners=100

```

```
maxCorners=100,
qualityLevel=0.01,
minDistance=5
)

next_points, status, error = cv2.calcOpticalFlowPyrLK(
    frame1,
    frame2,
    points,
    None
)

# Calculate average sparse flow error
sparse_error = np.mean(error)

print("Sparse Optical Flow Error:", sparse_error)

# -----
# Dense Optical Flow
# -----

flow = cv2.calcOpticalFlowFarneback(
    frame1,
    frame2,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

# Calculate motion magnitude
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

dense_magnitude = np.mean(magnitude)

print("Dense Optical Flow Average Magnitude:", dense_magnitude)

# -----
# Display Frames
# -----

plt.figure(figsize=(10, 4))

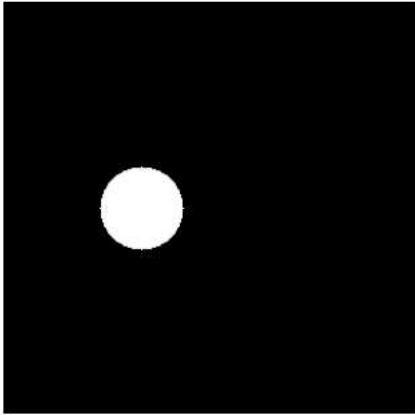
plt.subplot(1, 2, 1)
plt.imshow(frame1, cmap="gray")
plt.title("Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(frame2, cmap="gray")
plt.title("Frame 2")
plt.axis("off")

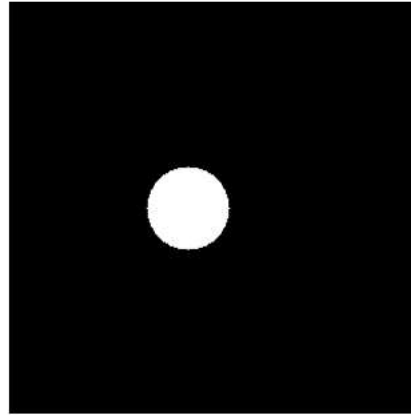
plt.show()
```

Sparse Optical Flow Error: 0.0015184645
 Dense Optical Flow Average Magnitude: 1.6284708

Frame 1



Frame 2



```
import cv2
import numpy as np

# Create GMM background subtractor
gmm = cv2.createBackgroundSubtractorMOG2()

# Example predicted and ground truth masks
ground_truth = np.array([1, 1, 0, 0, 1, 0])
prediction = np.array([1, 0, 1, 0, 1, 0])

FP = np.sum((prediction == 1) & (ground_truth == 0))
FN = np.sum((prediction == 0) & (ground_truth == 1))

negatives = np.sum(ground_truth == 0)
positives = np.sum(ground_truth == 1)

fpr = FP / negatives
fnr = FN / positives

print("False Positive Rate:", fpr)
print("False Negative Rate:", fnr)
```

False Positive Rate: 0.3333333333333333
 False Negative Rate: 0.3333333333333333

```
import cv2
import numpy as np

# Example errors for different window sizes
window_sizes = [4, 8, 16, 32]

errors = []

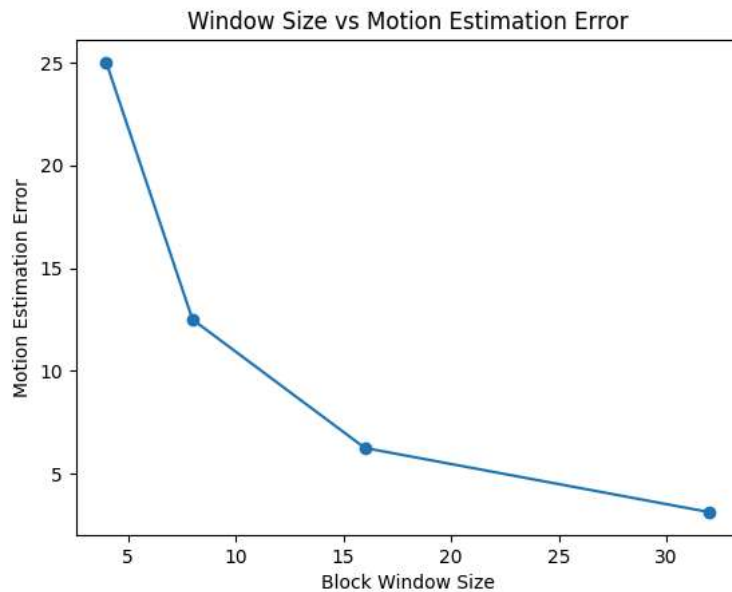
for size in window_sizes:
    # Example simulated motion estimation error
    error = 100 / size
    errors.append(error)

    print("Window Size:", size,
          "Motion Estimation Error:", error)

# Plot
import matplotlib.pyplot as plt

plt.plot(window_sizes, errors, marker="o")
plt.xlabel("Block Window Size")
plt.ylabel("Motion Estimation Error")
plt.title("Window Size vs Motion Estimation Error")
plt.show()
```

Window Size: 4 Motion Estimation Error: 25.0
 Window Size: 8 Motion Estimation Error: 12.5
 Window Size: 16 Motion Estimation Error: 6.25
 Window Size: 32 Motion Estimation Error: 3.125



```
import numpy as np

# Stereo camera parameters
focal_length = 700
baseline = 0.1

# Example disparity values
disparity = np.array([20, 25, 30, 35])

# Calculate estimated depth
estimated_depth = (focal_length * baseline) / disparity

# Ground truth depth
ground_truth_depth = np.array([3.5, 2.8, 2.3, 2.0])

# Mean Absolute Error
mae = np.mean(
    np.abs(estimated_depth - ground_truth_depth)
)

print("Estimated Depth:", estimated_depth)
print("Ground Truth Depth:", ground_truth_depth)
print("Mean Absolute Error:", mae)
```

```
Estimated Depth: [      3.5      2.8  2.3333      2]
Ground Truth Depth: [      3.5      2.8      2.3      2]
Mean Absolute Error: 0.0083333333333333415
```

```
import numpy as np
import matplotlib.pyplot as plt

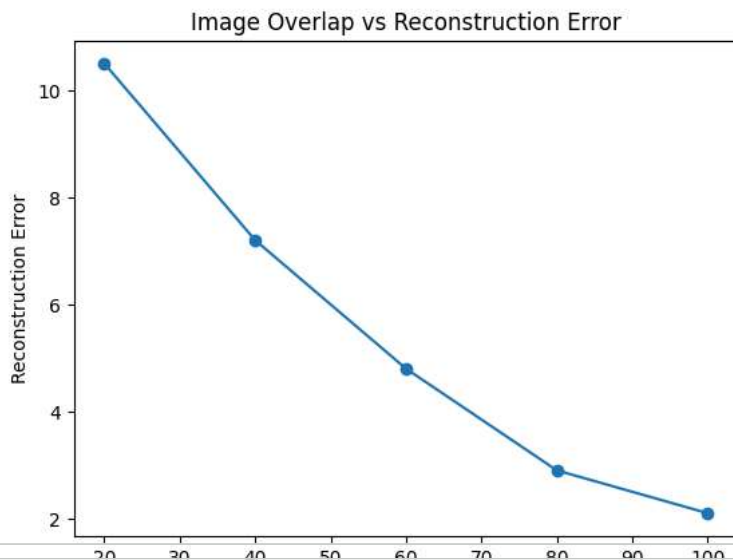
# Image overlap percentages
overlap = [20, 40, 60, 80, 100]

# Example reconstruction errors
error = [10.5, 7.2, 4.8, 2.9, 2.1]

plt.plot(overlap, error, marker="o")

plt.xlabel("Image Overlap (%)")
plt.ylabel("Reconstruction Error")
plt.title("Image Overlap vs Reconstruction Error")

plt.show()
```



```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np

# Ground truth
y_true = [0, 1, 2, 0, 1, 2, 0, 1, 2]

# Model A predictions
model_A = [0, 1, 2, 0, 1, 1, 0, 1, 2]

# Model B predictions
model_B = [0, 1, 1, 0, 2, 2, 0, 1, 2]

cm_A = confusion_matrix(y_true, model_A)
cm_B = confusion_matrix(y_true, model_B)

print("Model A Confusion Matrix:")
print(cm_A)

print("\nModel B Confusion Matrix:")
print(cm_B)

print("\nModel A Accuracy:",
      accuracy_score(y_true, model_A))

print("Model B Accuracy:",
      accuracy_score(y_true, model_B))
```

Model A Confusion Matrix:

```
[[3 0 0]
 [0 3 0]
 [0 1 2]]
```

Model B Confusion Matrix:

```
[[3 0 0]
 [0 2 1]
 [0 1 2]]
```

Model A Accuracy: 0.8888888888888888

Model B Accuracy: 0.7777777777777778